



Indian Liver Patient
Dataset.csv

Dataset used:

Gender of patient is encoded using Label Encoder

13 duplicates values were dropped and 4 null values of alkphos were changed. The project aims to find if the patient is a liver patient based on the input values from user.

Since the project expects a binary value with 1 implying the patient is a liver patient and 0 implying healthy live, Logistic Regression has been used. The trained model has an accuracy of

Accuracy: 0.7192982456140351

```
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import accuracy_score
import numpy as np
from sklearn.preprocessing import StandardScaler
import joblib

label_encoder=LabelEncoder()
data=pd.read_csv("Indian Liver Patient Dataset.csv")
data=data.drop_duplicates()
data["alkphos"] = data["alkphos"].fillna(data["alkphos"].mode()[0])
data["gender_encoded"]=label_encoder.fit_transform(data["gender"])
x=data.drop(columns=["gender","is_patient"])
y=data[["is_patient"]]
x_scaler=StandardScaler()

x_scaled=x_scaler.fit_transform(x)

x_train,x_test,y_train,y_test=train_test_split(x_scaled,y,test_size=0.2,random_state=42)
```

```

model=LogisticRegression()
model.fit(x_train,y_train)
y_pred=model.predict(x_test)
print("Accuracy:",accuracy_score(y_test,y_pred))
joblib.dump(model, 'modl.pkl')
joblib.dump(x_scaler,'x_scaler.pkl')
joblib.dump(label_encoder,'gender_encoded.pkl')

print(data.duplicated().sum())
print(data.isnull().sum())

```

Output:

```

Accuracy: 0.7192982456140351
0
age                0
gender              0
tot_bilirubin      0
direct_bilirubin   0
tot_proteins       0
albumin            0
ag_ratio           0
sgpt               0
sgot               0
alkphos            0
is_patient         0
gender_encoded     0
dtype: int64

```

```

import joblib
import numpy as np
modl=joblib.load('modl.pkl')
x_scaler=joblib.load('x_scaler.pkl')
gender_encoder=joblib.load('gender_encoded.pkl')
age=int(input("Enter age of patient:"))

gender= input("Enter Sex: ").strip().lower()

```

```

gender_encoded=gender_encoder.fit_transform([gender])[0]
tot_bilirubin=float(input("Enter total bilirubin count:"))
direct_bilirubin=float(input("Enter direct bilirubin count:"))
tot_proteins=float(input("Enter total proteins count:"))
albumin=float(input("Enter total albumin count:"))
ag_ratio=float(input("Enter ag_ratio:"))
sgpt=float(input("Enter sgpt count:"))
sgot=float(input("Enter sgot count:"))
alkphos=float(input("Enter alkphos count:"))

features =
np.array([[age,tot_bilirubin,direct_bilirubin,tot_proteins,albumin,ag_ratio,sgpt,sgot,alkphos,gender_
encoded]])

features_scaled = x_scaler.transform(features)

prediction = modl.predict(features_scaled)

if prediction[0]==1:
    print("Liver Patient")
else:
    print("Healthy liver")

```



```

PS C:\Users\Antony\My Python Practices\MLandAI> & C:\Users\Antony\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/Antony/My Python Practices/MLandAI/LiverPatientPred.py"
Enter age of patient:57
Enter Sex: Female
Enter total bilirubin count:4.7
Enter direct bilirubin count:5.7
Enter total proteins count:8.6
Enter total albumin count:3.7
Enter ag_ratio:1.2
Enter sgpt count:3.4
Enter sgot count:2.6
Enter alkphos count:4.7
C:\Users\Antony\AppData\Local\Programs\Python\Python313\Lib\site-packages\sklearn\utils\validation.py:2739: UserWarning: X does not have valid feature names, but StandardScaler was fitted w
ith feature names
  warnings.warn(

```

Project Reflection

Working on a liver disease prediction project showed me firsthand how machine learning can tackle real healthcare problems. I learned that building a model isn't only about choosing an algorithm — it begins with getting the data ready and really understanding it. Using patient details like age, gender, bilirubin levels, and other medical measures, I tried to predict who might have liver disease.

The hardest part was preprocessing. Dealing with missing values, converting categorical fields, and making sure the data used for training and prediction matched exactly took a lot of care. I also ran into several coding bugs — issues with feature scaling, label encoding, and loading saved models — and fixing those improved my debugging skills and made me more careful about implementation details.

Through the project I picked up essential machine-learning skills: data cleaning, feature scaling, creating train/test splits, building classifiers, and evaluating model performance. More than technical knowledge, the experience gave me confidence in applying ML to real problems and sharpened my programming and analytical abilities.